

INTERNBOT

LLM Selection & Evaluation Strategy

AI Feature Strategy · System Prompt Design · Benchmark Framework · Model Recommendation

Version 1.1 | April 2026 | Confidential — Internal Use

1. Executive Summary

This document is the definitive technical and strategic plan for selecting, benchmarking, and operating the Large Language Models (LLMs) that power Internbot's three AI features. It is authored by the designated LLM & AI expert and constitutes the primary evidence base for all model selection decisions.

Internbot ships three AI-powered capabilities:

- AI Contract Checker — structured Yes/No classification of internship contracts against compliance rules, with concern extraction and summary.
- AI Self-Sourced Job Checker — classification of student-submitted job descriptions against RMIT eligibility criteria.
- FAQ Bot — a Retrieval-Augmented Generation (RAG) chatbot answering student and coordinator questions from an approved knowledge base, built serverlessly on Firebase.

Recommended primary stack: GPT-4o (via RMIT proxy) for the checkers; Gemini 2.0 Flash Lite for high-volume FAQ generation. Self-hosted Llama-3.1-70B on the Garage 4× RTX 5090 workstation is benchmarked as a cost-zero, full data-sovereignty fallback. Final model selection is governed by the quantitative benchmark framework in Section 6.

2. Context & Feature Requirements

2.1 Platform Context

Internbot (Firebase + Cloud Functions v2 + Firestore) manages the end-to-end student internship workflow: profile completion → position description review → job offer review → semester selection. All AI features are stateless Cloud Functions; no AI-generated content is persisted to Firestore. Volume estimates for MVP:

- ~400–800 internship submissions per semester per cohort.
- ~2,000–5,000 FAQ queries per semester (spikes around enrolment deadlines).
- Peak concurrency: low to moderate — university workflow, not consumer-scale.

2.2 Feature Requirements

Feature A — AI Contract Checker

Requirement	Detail
Input	Raw contract text (PDF-extracted, up to ~8,000 tokens)
Output	{ decision: Yes/No, confidence: 0–1, concerns: string[], summary: string }
Rules source	Coordinator-defined compliance ruleset encoded in versioned system prompt
Accuracy target	>= 90% precision on Yes; >= 85% recall on No
Latency SLO	P50 < 10s, P95 < 18s

Requirement	Detail
Data sensitivity	High — contains PII (student name, employer, salary). Confirm data handling policy before sending to cloud APIs.
Cost target	< \$0.05 USD per call

Feature B — AI Self-Sourced Job Checker

Requirement	Detail
Input	Job description text (student form, up to ~3,000 tokens)
Output	{ decision: Yes/No, confidence: 0–1, concerns: string[], summary: string }
Rules source	Same ruleset pattern as Contract Checker (separate prompt)
Accuracy target	>= 88% precision, >= 85% recall
Latency SLO	P50 < 8s, P95 < 15s
Data sensitivity	Medium — job descriptions are public-facing but include student identity context
Cost target	< \$0.03 USD per call

Feature C — FAQ Bot (RAG, Serverless on Firebase)

Requirement	Detail
Input	Natural language question (student or coordinator)
Output	{ answer: string, sources: [{title, section}], disclaimer: string }
Knowledge base	RMIT internship policy, WIL handbook, coordinator FAQs. Chunked + vectorised.
RAG pipeline	Embedding → vector search (Firestore vector search) → context injection → LLM generation
Accuracy target	Answer faithfulness >= 85%; hallucination rate < 5%
Latency SLO	P50 < 5s, P95 < 9s (conversational feel)
Data sensitivity	Low to medium — questions may contain student context; answers are policy-derived
Cost target	< \$0.01 USD per query; HTTP caching for repeated queries

3. Available Model Landscape

3.1 RMIT Proxy — OpenAI-Compatible API

Alias	Actual Model	Intended Use	Capability Tier
BIG_MODEL	OpenAI GPT-5.4	Complex reasoning, document analysis	Highest
MIDDLE_MODEL	OpenAI GPT-4o	Sonnet-class tasks — checker primary	High
SMALL_MODEL	OpenAI GPT-4o	Haiku-class tasks — FAQ fallback	High

- Data residency: governed by RMIT's OpenAI enterprise agreement. Confirm with IT Security whether PII (student names, employer details in contracts) is permissible before sending contract text.
- Cost: proxied through university budget — confirm billing model with Alessio / IT.
- Availability: shared institutional resource; monitor rate-limit headroom.

3.2 Google Gemini 2.0 Flash Lite

- Positioning: Google's cost-efficient, high-throughput model optimised for low latency. Excellent FAQ generation candidate.
- Context window: 1M tokens — ample for any contract or FAQ use case.
- Pricing (April 2026): ~\$0.0003/1K input tokens, ~\$0.0006/1K output tokens — among the cheapest frontier-adjacent options available.
- Data residency: Google Cloud; available in australia-southeast1 (Sydney). Configure the API project to this region to satisfy any data-sovereignty requirements.
- Multimodality: supports image input natively — relevant if PDF pages are passed directly rather than extracted text.
- Firebase integration: available via the Firebase AI Logic SDK (formerly Vertex AI for Firebase), enabling direct calls from Cloud Functions with zero additional auth setup.

3.3 Self-Hosted Models — Garage Workstation (4× NVIDIA RTX 5090)

Attribute	Detail
Hardware	4× RTX 5090 (~24 GB GDDR7 each ≈ 96 GB total VRAM)
Recommended model	Llama-3.1-70B-Instruct (fits at Q4 quantisation with 4-way tensor parallelism)
Alternate models	Mistral-7B-Instruct (trivial, baseline), Qwen2.5-72B, Llama-3.3-70B
Serving stack	vLLM with PagedAttention, OpenAI-compatible REST API, tensor parallelism across 4 GPUs
Throughput	~15–30 tokens/sec at 70B Q4 — adequate for our submission volume
Marginal cost	Zero (sunk hardware). Electricity + maintenance only.
Data residency	100% on-premises — strongest privacy posture, zero data leaves university.
Limitations	Team responsible for uptime. No SLA. Cold-start risk on reboot.

⚠ Self-Hosting Operational Warning

- The self-hosted option requires someone to maintain the vLLM service, apply model updates, and keep the GPU workstation online.
- Treat it as a secondary/fallback tier unless dedicated ML Ops capacity exists.
- Run it in parallel during the benchmark phase to generate comparative data before committing to it as primary.

4. System Prompt Strategy

4.1 Prompt Engineering Principles

- All checker prompts must elicit strict JSON output only — no prose preamble, no markdown code fences in production responses.
- Use chain-of-thought via a scratchpad field: instruct the model to reason through each rule step-by-step internally before emitting a final decision. This materially improves accuracy on borderline cases.
- Few-shot examples are mandatory. Embed at least 3 clearly-passing and 3 clearly-failing labelled examples directly in the system prompt. Ground each example in real-world contract language.
- Define the exact JSON output schema in the prompt. Specify field names, types, allowed values, and constraints. The model must be told to never deviate.
- Version every prompt in source control (/prompts/contract-checker-v1.0.0.txt). Tag each benchmark run with the prompt version that produced it.
- Coordinator sign-off is required before any rule addition or removal from checker prompts. Changes are semantic, not cosmetic.

4.2 Contract Checker — Full System Prompt

The following is the production-ready system prompt for the AI Contract Checker. It encodes all compliance rules, chain-of-thought reasoning, few-shot examples, and strict output schema.

```
You are a strict internship contract compliance auditor for Internbot, the RMIT
internship management platform. Your sole function is to determine whether a
submitted
internship contract PASSES (Yes) or FAILS (No) RMIT's compliance requirements.
```

```
You must be conservative: when uncertain, lean toward No and surface a concern.
A false positive (approving a non-compliant contract) has serious consequences.
A false negative (flagging a compliant contract) is recoverable by the coordinator.
```

```
COMPLIANCE RULES (all must pass for decision: Yes)
```

RULE 1 — AWARD CLASSIFICATION

```
The employment award or classification must be a recognised IT, Software
Engineering,
Computer Science, or closely related Engineering classification.
```

FAIL if: award is Healthcare (e.g. Health Professional Award), Retail, Hospitality, Clerical, or any classification unrelated to computing or engineering disciplines.
AMBIGUOUS: 'IT Support' roles must also satisfy Rule 5 (technical depth).

RULE 2 – MINIMUM HOURS

The contract must specify or clearly imply a minimum of 240 hours of industry placement work.
FAIL if: hours are not stated, stated hours are below 240, or phrasing is so vague (e.g. 'part-time as needed') that 240 hours cannot be reasonably inferred.
PASS if: '240 hours', '6 weeks full-time', or equivalent clear statement is present.

RULE 3 – NAMED SUPERVISOR

The contract must name a specific workplace supervisor with at minimum a name and either an email address or phone number.
FAIL if: supervisor section is blank, states 'TBD', lists only a department name, or provides no contact method.

RULE 4 – STUDENT AND EMPLOYER IDENTITY MATCH

The student's full name and the employer's legal entity name in the contract must match the information submitted in the internship form.
FAIL if: student name is different, employer name differs materially (e.g. a subsidiary is named when the parent company was declared), or either is absent.

RULE 5 – SUBSTANTIVE TECHNICAL WORK

The described role must involve substantive technical tasks directly relevant to a computing or engineering program.
PASS examples: software development, system design, data analysis, network engineering, cybersecurity, ML/AI research, cloud infrastructure.
FAIL examples: data entry, reception duties, sales support, administrative tasks even if performed at a technology company.
BORDERLINE: roles mixing technical and non-technical tasks – flag a concern and let the coordinator decide unless clearly majority non-technical.

RULE 6 – CONTRACT VALIDITY

The contract must carry a valid start date that has not yet passed, and must be signed (digitally or physically) by both the employer and student.
FAIL if: start date is in the past at time of submission, or the contract shows no evidence of mutual signing.

FEW-SHOT EXAMPLES

EXAMPLE 1 – PASS

Contract excerpt: 'This agreement is between Example Pty Ltd and Alex Smith (s3456789) for the role of Junior Software Developer under the IT Award 2020. Alex will complete 240 hours of software development work across 6 weeks... Supervised by Jane Doe (jane.doe@example.com)...'
{
 "scratchpad": "Rule1: IT Award 2020 – PASS. Rule2: 240 hours stated – PASS. Rule3: Jane Doe with email – PASS. Rule4: Alex Smith matches form – PASS. Rule5: Software development – PASS. Rule6: Future start, signed – PASS.",
 "decision": "Yes", "confidence": 0.97,
 "concerns": [], "summary": "Contract fully compliant across all six rules."
}

EXAMPLE 2 – FAIL (wrong award)

Contract excerpt: 'This agreement is for a Health Information Assistant role under the Health Professionals Award 2010...'

```
{
  "scratchpad": "Rule1: Health Professionals Award – FAIL. This is a healthcare
classification, not IT or Engineering. No further rules need to pass.",
  "decision": "No", "confidence": 0.99,
  "concerns": ["Award classification is Health Professionals Award (healthcare),
not a recognised IT or Engineering award."],
  "summary": "Contract fails Rule 1: award classification is healthcare, not IT."
}

EXAMPLE 3 – FAIL (insufficient hours)
Contract excerpt: '...the student will work part-time on an as-needed basis...'
{
  "scratchpad": "Rule2: No specific hours stated. Part-time as-needed is too
vague to infer 240 hours. FAIL.",
  "decision": "No", "confidence": 0.93,
  "concerns": ["Minimum 240 hours not specified. Part-time as-needed is
insufficient."],
  "summary": "Contract fails Rule 2: required 240-hour minimum is not established."
}
```

OUTPUT SCHEMA (return ONLY this JSON – no other text)

```
{
  "scratchpad": "<evaluate each rule explicitly, one sentence per rule>",
  "decision": "Yes" | "No",
  "confidence": "<0.0 to 1.0, reflecting certainty in the decision>",
  "concerns": ["<specific concern 1>", "<specific concern 2>"],
  "summary": "<one sentence explaining the overall outcome>"
}
```

Rules: decision is Yes only if ALL six rules pass. Any single rule failure = No.
concerns must be empty [] if decision is Yes.
concerns must contain at least one entry if decision is No.
confidence below 0.75 means edge case – always surface a concern.

4.3 Self-Sourced Job Checker — Full System Prompt

The Job Checker uses a lighter ruleset focused on eligibility rather than contract formality.

```
You are an internship eligibility reviewer for Internbot, the RMIT internship
management platform. Your task is to assess whether a self-sourced job description
submitted by a student is ELIGIBLE (Yes) or INELIGIBLE (No) for RMIT's Work-
Integrated
Learning (WIL) internship program.
```

ELIGIBILITY RULES

RULE 1 – TECHNICAL DISCIPLINE ALIGNMENT

```
The role must be directly relevant to software engineering, computer science,
information technology, or a closely related computing discipline.
FAIL: IT helpdesk only, data entry, generic 'digital marketing', sales.
PASS: software development, data science, DevOps, cybersecurity, AI/ML, UX/UI
design with a technical component, cloud engineering, network administration.
```

RULE 2 – MINIMUM PLACEMENT DURATION

The role must offer a placement of at least 240 hours (approx. 6 full-time weeks).

Part-time placements are acceptable if they accumulate to 240 hours.

FAIL if: role is clearly casual/ad-hoc, is a short project under 4 weeks, or no duration information is present and it cannot be reasonably inferred.

RULE 3 – SUPERVISION STRUCTURE

The student must have access to a named or clearly implied workplace supervisor who can provide regular feedback and sign off on their work.

FAIL if: the role is fully autonomous with no supervision structure described, or it is a freelance/contractor engagement with no host organisation.

RULE 4 – PAID OR FORMALLY STRUCTURED

The role must be either: (a) a paid position, or (b) an unpaid placement with a formal WIL agreement in place (e.g. sponsored by the university).

FAIL if: the role is unpaid with no evidence of a formal placement structure (e.g. informal volunteer work at a startup with no agreement).

RULE 5 – LEGITIMATE EMPLOYER

The employer must appear to be a legitimately operating organisation (registered business, university, government body, or established NFP).

FAIL if: employer is anonymous, is a personal trading name with no ABN context, or the submission contains no employer information.

FEW-SHOT EXAMPLES

EXAMPLE 1 – ELIGIBLE

Submission: 'Junior React Developer at Acme Fintech Pty Ltd. 6-month full-time internship. Supervised by a senior engineer. Paid at market rates.'

```
{
  "scratchpad": "R1: React/software dev – PASS. R2: 6 months > 240hrs – PASS.
  R3: Senior engineer supervisor – PASS. R4: Paid – PASS. R5: Acme Fintech Pty Ltd
– PASS.",
  "decision": "Yes", "confidence": 0.96,
  "concerns": [], "summary": "Role is a fully eligible paid software internship."
}
```

EXAMPLE 2 – INELIGIBLE (not technical)

Submission: 'Social media content creator at TechBrand Co. Posting Instagram content and managing influencer partnerships. 3 months part-time.'

```
{
  "scratchpad": "R1: Social media content creation is not a computing discipline –
FAIL.",
  "decision": "No", "confidence": 0.95,
  "concerns": ["Role is social media/marketing, not a computing or IT
discipline."],
  "summary": "Ineligible: role does not align with a computing or engineering
program."
}
```

EXAMPLE 3 – INELIGIBLE (insufficient duration)

Submission: 'Two-week sprint project for a startup. Unpaid. No formal agreement.'

```
{
  "scratchpad": "R2: 2 weeks is far below 240 hours – FAIL. R4: Unpaid with no
formal agreement – FAIL.",
  "decision": "No", "confidence": 0.98,
  "concerns": ["Duration of 2 weeks is below the 240-hour minimum.", "Unpaid with
no formal WIL agreement."],
  "summary": "Ineligible: insufficient duration and no formal placement structure."
}
```

OUTPUT SCHEMA (return ONLY this JSON – no other text)

```
{
  "scratchpad": "<evaluate each rule explicitly, one sentence per rule>",
  "decision": "Yes" | "No",
  "confidence": <0.0 to 1.0>,
  "concerns": [<"concern 1">, ...],
  "summary": "<one sentence outcome>"
}
```

decision is Yes only if ALL five rules pass. Any single rule failure = No.

4.4 FAQ Bot — System Prompt

The FAQ bot uses a two-stage approach: a retrieval stage (embedding + vector search) followed by a grounded generation stage. The system prompt governs the generation stage.

You are the Internbot FAQ assistant, a helpful and accurate support agent for RMIT's internship program. You answer questions from students and coordinators about the internship process, eligibility, policy, and procedures.

CRITICAL GROUNDING RULES – YOU MUST FOLLOW THESE

1. Answer ONLY using the CONTEXT provided below. Do not use prior knowledge, general assumptions, or information not present in the context.
2. If the answer cannot be found in the context, respond with EXACTLY:
'I don't have enough information to answer that from the available policy documents. Please contact your course coordinator directly for guidance.'
Do NOT attempt to answer from general knowledge.
3. Cite the specific document and section that supports your answer.
Format: [Source: <document title>, <section>]
4. Do not speculate about future policy changes, coordinator decisions, or outcomes for specific students. Your role is to explain existing policy only.
5. Keep answers concise and student-friendly. Use plain English.
Avoid bureaucratic jargon where possible.

CONTEXT (retrieved from approved RMIT policy documents)

{{RETRIEVED_CHUNKS}}

OUTPUT FORMAT

Respond with valid JSON only:

```
{
  "answer": "<your grounded answer in plain English>",
  "sources": [{ "title": "<doc title>", "section": "<section name>" }],
  "confidence": <0.0 to 1.0, how well the context supports the answer>,
  "answered_from_context": true | false
}
```

```
}
```

```
If answered_from_context is false, answer must be the refusal message above,  
sources must be [], and confidence must be 0.0.
```

4.5 Prompt Versioning & Governance

Practice	Implementation
Version control	Git-tracked in /prompts/ directory. Semantic versioning (v1.0.0 → v1.1.0 for rule tweaks, v2.0.0 for structural changes).
Changelog	Each prompt version documents what changed and why (e.g. 'v1.1.0 — tightened Rule 1 after false positive batch on IT support roles').
Rollback	Prompt version stored with each benchmark log. Can revert independently of code.
Runtime config	Cloud Function reads prompt version from a Firestore config document (prompt_config/{feature}). Swappable without deployment.
Coordinator approval	Required before any rule addition/removal. Changes are semantic, not cosmetic.
A/B testing	Shadow flag in Cloud Function sends % of traffic to a new prompt version in parallel. Compare metrics before promoting.

5. Data Strategy & Firebase RAG Architecture

5.1 Ground-Truth Dataset Construction

A labelled dataset is the non-negotiable foundation of every quantitative evaluation. Without it, model selection is guesswork.

5.1a Contract Checker Dataset

- Source: coordinate with Alessio to obtain 20–50 real historical contracts. Anonymise PII (replace names with placeholders) before use in benchmarks.
- Labelling: each contract labelled Yes/No + which rule(s) triggered + confidence by a domain expert (coordinator). This becomes ground truth.
- Augmentation: if real contracts are scarce, generate synthetic contracts using a Big model with deliberate rule violations. All synthetic samples must be reviewed manually.
- Target size for MVP benchmark: 30 contracts minimum (15 Yes / 15 No). 60+ for production confidence.
- Stratification: ensure coverage of edge cases — borderline health-IT contracts, contracts missing only one field, ambiguous award classifications, roles with mixed technical/admin duties.

5.1b Job Checker Dataset

- Source: historic self-sourced job submissions (anonymised). Generate synthetic entries for rare categories (e.g. borderline IT support, creative-tech roles).

- Target size: 40 job descriptions (20 pass / 20 fail).
- Edge cases: IT-adjacent roles at non-tech firms, roles with ambiguous technical depth, unpaid placements with no formal agreement.

5.1c FAQ Bot Dataset

- Source: collate the top 30–50 questions coordinators actually receive (email archives, student messages). Write reference answers grounded in policy documents.
- Format: { question, reference_answer, source_document, source_section } tuples.
- Faithfulness test: include 10 questions that are NOT answerable from the knowledge base to validate refusal behaviour.

5.2 Firebase-Native RAG Architecture

All RAG infrastructure runs within the Firebase/Google Cloud ecosystem, keeping the stack serverless and aligned with Internbot's existing architecture.

Layer	Component	Firebase/GCP Service	Notes
Document ingestion	PDF/DOCX policy docs uploaded	Cloud Storage bucket	Triggered on upload via onObjectFinalized Cloud Function
Text extraction	PDF → plain text chunks	Cloud Function (pdfjs / pdf-parse)	Chunk size: ~500 tokens, 10% overlap. Metadata tagged per chunk.
Embedding generation	Text → dense vector	Gemini text-embedding-004 via Firebase AI Logic SDK	768-dimensional embeddings. Batch-processed on ingestion.
Vector storage	Chunk vectors + metadata	Firestore vector search (native)	Firestore now supports approximate nearest-neighbour queries. No external vector DB needed.
Query embedding	Student question → vector	Same Gemini embedding model	Called in-request within the FAQ Cloud Function.
Retrieval	Top-K chunk lookup	Firestore .findNearest() query	K=5 by default. Filtered by document_status == 'active'.
Generation	Chunks + question → answer	Gemini Flash Lite / GPT-4o (fallback)	System prompt from Section 4.4 injected with retrieved chunks.
Caching	Repeated query deduplication	Firestore faq_cache collection (TTL 24h)	Normalised question hash as document ID. Cache-first on every request.

Layer	Component	Firebase/GCP Service	Notes
Response	Structured JSON to client	Cloud Function → REST API	disclaimer field appended server-side, never left to the LLM.

5.3 Knowledge Base Document Plan

Document	Format	Chunking Strategy	Update Frequency
RMIT WIL Internship Policy (full handbook)	PDF	Chapter-level chunks (~500 tokens) with 50-token overlap	Annual (each academic year)
Coordinator FAQ Q&A pairs	Markdown / DOCX	Individual Q&A pairs as atomic chunks	As needed; coordinator-owned
Program eligibility rules (BP096, BP347, BP349)	DOCX	Per-program rule sets, one chunk per rule group	On program changes
Semester dates & enrolment windows	Structured config	Key-value pairs stored as Firestore docs, not vectorised	Each semester
Common rejection reasons (coordinator notes)	Markdown	Scenario-based chunks of 200–300 tokens	Monthly review
WIL course outlines (INTE2710, 2711, 2708)	PDF	Section-level chunks	Annual

5.4 Firestore Vector Search Setup

Firestore's native vector search (GA as of 2025) eliminates the need for an external vector database like Pinecone, keeping all data within the Firebase project boundary.

```
// Firestore collection structure for FAQ knowledge base
faq_chunks/{chunkId} {
  text:           string,           // the chunk text
  embedding:       vector(768),     // Gemini text-embedding-004 output
  document_title: string,
  section:         string,
  document_status: 'active' | 'archived',
  version_date:    timestamp,
  chunk_index:     number           // position within source document
}

// Retrieval query inside Cloud Function (Node.js)
const queryEmbedding = await embedText(userQuestion);
const results = await db.collection('faq_chunks')
  .where('document_status', '==', 'active')
  .findNearest('embedding', queryEmbedding, {
    limit: 5,
    distanceMeasure: 'COSINE'
  })
  .get();

const chunks = results.docs.map(d => d.data().text);
// → inject chunks into FAQ system prompt as {{RETRIEVED_CHUNKS}}
```

6. Benchmark & Evaluation Framework

6.1 Evaluation Metrics by Feature

Contract & Job Checkers (Classification)

Metric	Formula / Method	Target	Rationale
Precision (Yes)	$TP / (TP + FP)$	$\geq 90\%$	False positives pass non-compliant contracts — highest risk
Recall (Yes)	$TP / (TP + FN)$	$\geq 80\%$	False negatives block valid students — recoverable by coordinator
F1 Score	$2 \times (P \times R) / (P + R)$	≥ 0.85	Balanced single-number comparison across models
Cohen's Kappa	Agreement beyond chance	≥ 0.75	Measures reliability vs. ground truth beyond random agreement
Confidence Calibration (ECE)	Expected Calibration Error	< 0.10	Confidence scores must reflect true probability of correctness
Concern Match Rate	% concerns overlapping GT concerns (semantic similarity ≥ 0.80)	$\geq 70\%$	Qualitative accuracy of explanation, not just the label
Rule Attribution Accuracy	% of runs identifying the correct failing rule	$\geq 80\%$	Validates that the scratchpad reasoning is correct, not just the label
Latency P50 / P95	Seconds per request	P50 $< 8s$ / P95 $< 15s$	Coordinator dashboard responsiveness
Cost per call	USD per inference (input + output tokens $\times$ price)	$< \$0.05$ (checker) / $\$0.03$ (job)	Budget sustainability
Parse Success Rate	% responses successfully parsed as valid JSON	$\geq 99\%$	Production reliability — JSON failures require retry or fallback

FAQ Bot (RAG Generation)

Metric	Measurement Method	Target
Answer Faithfulness	LLM-as-judge: does the answer use only retrieved context? (GPT-4o judge)	$\geq 85\%$
Answer Relevance	Cosine similarity between answer embedding and question embedding	≥ 0.80
Context Recall	% of ground-truth facts present in top-5 retrieved chunks	$\geq 75\%$

Metric	Measurement Method	Target
Refusal Rate (unanswerable Qs)	% of out-of-KB questions correctly refused (answered_from_context == false)	>= 90%
Hallucination Rate	% answers containing facts absent from all retrieved chunks	< 5%
Source Citation Accuracy	% of cited sources that actually appear in retrieved chunks and support the claim	>= 90%
Latency P50 / P95	Seconds (embedding + retrieval + generation)	P50 < 4s / P95 < 8s
Cost per query	USD per FAQ call (embedding + generation)	< \$0.01
Cache Hit Rate	% requests served from Firestore cache	>= 30% at steady state

6.2 Concrete Benchmark Test Suite

The following are the specific test cases that must be run against every candidate model. These are not abstract metrics — they are actual inputs with known correct outputs that directly measure model behaviour on Internbot-relevant scenarios.

6.2a Contract Checker Test Cases

Test ID	Description	Key Features	Expected Decision	Key Metric Tested
CC-001	Fully compliant IT contract — all 6 rules pass clearly.	IT Award, 240hrs stated, named supervisor, matching names, technical role, signed.	Yes (conf > 0.90)	Precision (Yes), Calibration
CC-002	Healthcare award — Rule 1 fails.	Health Professionals Award 2010, 240hrs, all else compliant.	No (conf > 0.92)	Rule 1 attribution, Recall
CC-003	Missing hours — Rule 2 fails.	IT Award, 'part-time as needed', named supervisor, all else compliant.	No (conf > 0.88)	Rule 2 attribution
CC-004	No supervisor contact — Rule 3 fails.	IT Award, 240hrs, supervisor name only (no email/phone), compliant otherwise.	No (conf > 0.85)	Rule 3 attribution
CC-005	Student name mismatch — Rule 4 fails.	Contract names 'Alexandra Smith', form submitted as 'Alex Smith' — surname differs.	No	Rule 4 — name sensitivity
CC-006	Data entry role at tech company — Rule 5 fails.	IT Award, 240hrs, all fields correct, role is 'Data Entry Officer at Google.'	No (conf > 0.90)	Rule 5 — company halo effect

Test ID	Description	Key Features	Expected Decision	Key Metric Tested
CC-007	Mixed technical and admin role — Rule 5 borderline.	Role is '70% software dev, 30% team admin support.'	Yes with concern	Concern generation on borderline
CC-008	Expired contract — Rule 6 fails.	Start date 3 months in the past, fully signed, all else correct.	No	Rule 6 — date handling
CC-009	Multiple rule failures.	Healthcare award, no hours, no supervisor.	No (conf > 0.97)	Multi-rule failure handling
CC-010	Ambiguous IT Support role.	IT Award, 240hrs, supervisor listed, role is 'IT Support Technician' — no further technical detail.	No or Yes with concern	Concern Match Rate, Calibration
CC-011	Pre-filled synthetic pass (used for latency benchmark).	Unambiguous pass contract, ~4,000 tokens.	Yes	Latency P50/P95, Cost
CC-012	Long contract — stress test (~8,000 tokens).	Full contract with appendices, all rules pass.	Yes	Latency P95 at max input

6.2b Job Checker Test Cases

Test ID	Description	Key Features	Expected Decision	Key Metric Tested
JC-001	Clear eligible role — all 5 rules pass.	Junior Software Developer, 6-month paid, senior engineer supervisor, Acme Pty Ltd.	Yes (conf > 0.90)	Precision (Yes)
JC-002	Social media role — Rule 1 fails.	Social media content creator at a tech company.	No (conf > 0.92)	Rule 1 — company halo
JC-003	Duration too short — Rule 2 fails.	2-week sprint project, technical role, paid.	No	Rule 2 — duration floor
JC-004	Fully autonomous freelance — Rule 3 fails.	Build an app as a solo contractor. No supervisor named.	No	Rule 3 — supervision requirement
JC-005	Unpaid, no formal agreement — Rule 4 fails.	Unpaid internship at a startup. No WIL agreement mentioned.	No	Rule 4 — unpaid detection
JC-006	Anonymous employer — Rule 5 fails.	Submission has no employer name or ABN. Just 'a local startup'.	No	Rule 5 — employer identity
JC-007	Data science role — eligible.	Data analyst / ML engineer role, 6-month paid, established employer.	Yes	Discipline breadth

Test ID	Description	Key Features	Expected Decision	Key Metric Tested
JC-008	IT helpdesk + some software tasks.	Role is 60% helpdesk, 40% scripting automation. Paid, 240hrs.	Borderline — Yes with concern or No	Borderline Rule 1, Concern Match
JC-009	Part-time accumulating to 240hrs.	Part-time (3 days/week for 5 months), technical, paid.	Yes	Rule 2 — part-time accumulation
JC-010	Latency / cost stress test.	Clear eligible role, ~2,500 tokens.	Yes	Latency P50/P95, Cost/call

6.2c FAQ Bot Test Cases

Test ID	Question	Expected Behaviour	Key Metric Tested
FAQ-001	How many hours do I need to complete for my internship?	Answer: 240 hours. Source: WIL handbook, Eligibility section.	Faithfulness, Citation Accuracy
FAQ-002	I haven't found an internship yet. What should I do?	Answer: outlines next steps (Career Hub, coordinator, self-sourcing). Source: FAQ doc.	Relevance, Faithfulness
FAQ-003	Can I defer my internship to next semester?	Answer: explains deferral policy from policy doc. Source cited.	Faithfulness, Context Recall
FAQ-004	How do I transfer from CS to Software Engineering?	Refusal: not in internship policy KB. Direct to coordinator.	Refusal Rate — out-of-KB
FAQ-005	What is the difference between INTE2710 and INTE2711?	Answer: INTE2710 is Internship 2a (first 240hr block), INTE2711 is Internship 2b. Source: WIL course outlines.	Faithfulness, Source Accuracy
FAQ-006	My contract was rejected. What are the most common reasons?	Answer: lists common rejection reasons from coordinator notes chunk.	Context Recall, Relevance
FAQ-007	Can I do my internship at my family's business?	Answer: policy on conflicts of interest / related-party internships.	Faithfulness — policy nuance
FAQ-008	What GPA do I need to do an internship?	Answer: GPA requirement from eligibility rules (3.0 or as specified). Source cited.	Faithfulness, Calibration
FAQ-009	Who is the best coordinator to talk to?	Refusal: personal coordinator assignments not in KB.	Refusal Rate — personal/dynamic
FAQ-010	What happens after my job offer is approved?	Answer: semester selection step. Source: WIL handbook, workflow section.	Faithfulness, Relevance

Test ID	Question	Expected Behaviour	Key Metric Tested
FAQ-011	Same question as FAQ-001 rephrased: 'How long does the placement need to be?'	Served from cache (same normalised hash) OR identical answer.	Cache Hit Rate, Consistency
FAQ-012	Can I complete my internship part-time?	Answer: yes, if hours accumulate to 240. Source cited.	Faithfulness — part-time policy
FAQ-013	Hallucination probe: 'Is there a \$5,000 completion bonus?'	Refusal or clear No — this fact does not exist in KB.	Hallucination Rate
FAQ-014	What courses do I enrol in alongside the internship?	Answer: INTE2708 Internship Reflection 1 runs alongside placement. Source: course outlines.	Context Recall, Citation Accuracy
FAQ-015	Latency stress: long rambling question (~300 words) about general internship anxiety.	Focused, grounded answer or polite refusal if not policy-relevant.	Latency P95, Faithfulness

6.3 Benchmark Execution Protocol

1. Prepare evaluation harness: a Python script that calls each model API, logs raw response + parsed JSON + latency. Results stored in structured JSON logs per run (one file per model × prompt version × feature).
2. Run each model against the full test suite 3 times with temperature=0 (or lowest available) to account for any residual stochasticity. Average metrics across runs.
3. For each test case, record: model name, model version, prompt version, test ID, ground truth, model decision, confidence, concerns list, latency_ms, input_tokens, output_tokens, parse_success, timestamp.
4. Compute all metrics from Section 6.1 per model. Produce the comparison table in Section 7.
5. Failure mode analysis: for every incorrect prediction, categorise failure type — wrong label, correct label but wrong confidence, missing concern, hallucinated concern, wrong rule attributed, JSON parse failure.
6. Human review of all borderline test cases (CC-007, CC-010, JC-008) by the coordinator. Their judgment overrides the automated metric for those cases.

6.4 Scoring Rubric for Model Selection

Dimension	Weight	Measurement
Classification Accuracy (F1)	35%	Average F1 across all Contract + Job Checker test cases
Latency (P95)	20%	Scaled: 1.0 if P95 < 10s, degrades linearly to 0 at 30s
Cost per 1,000 calls	20%	Scaled: 1.0 if < \$5/1K, degrades to 0 at \$50/1K

Dimension	Weight	Measurement
Hallucination / Faithfulness	15%	Inverse hallucination rate (FAQ); Concern Match Rate (checkers)
Data Residency / Privacy	10%	On-premises = 1.0, AU-region cloud = 0.8, overseas = 0.5

A model must score ≥ 0.75 composite AND pass hard gates (Precision(Yes) $\geq 88\%$, Latency P95 < 20s, Parse Success $\geq 99\%$) to be eligible for production.

7. Pre-Benchmark Model Comparison (Prior Estimates)

The following estimates are based on published benchmarks, model cards, and known characteristics. They will be replaced with empirical results from Section 6.2 test suite execution.

Model	Est. F1 (Checker)	Est. Faithfulness (FAQ)	P95 Latency	Cost/1K calls	Data Residency	Est. Composite
GPT-5.4 (BIG — RMIT proxy)	0.93–0.96	0.90–0.94	10–18s	~\$8–15	US/EU (RMIT MSA)	0.82
GPT-4o (MIDDLE — RMIT proxy)	0.90–0.93	0.87–0.92	6–12s	~\$3–6	US/EU (RMIT MSA)	0.86
Gemini 2.0 Flash Lite	0.84–0.89	0.85–0.90	2–5s	~\$0.3–0.8	AU-SE1 (Sydney)	0.83
Llama-3.1-70B (Self-hosted)	0.82–0.88	0.80–0.87	8–20s	\$0 (marginal)	On-premises ✓	0.80



Preliminary Recommendation (Subject to Benchmark Validation)

- Contract Checker: GPT-4o via RMIT proxy. Best accuracy/latency balance. Confirm PII data handling policy first.
- Job Checker: Gemini 2.0 Flash Lite. Likely meets accuracy targets at a fraction of the cost. Benchmark will confirm.
- FAQ Bot: Gemini 2.0 Flash Lite for generation, native Firebase AI Logic SDK, Firestore vector search for retrieval. Zero external dependencies.
- Self-hosted Llama-3.1-70B: benchmark in parallel. If F1 ≥ 0.86 , becomes primary option for both checkers (zero marginal cost, full data control, strongest privacy posture).

8. Cost Model & Trade-Off Analysis

8.1 Semester Cost Projections

Scenario	Feature	Volume/Sem	Est. Cost/Call	Sem. Total (USD)	Sem. Total (AUD ~)
GPT-4o Primary	Contract Checker	600 calls	\$0.04	\$24	~\$37

Scenario	Feature	Volume/Sem	Est. Cost/Call	Sem. Total (USD)	Sem. Total (AUD ~)
GPT-4o Primary	Job Checker	600 calls	\$0.025	\$15	~\$23
GPT-4o Primary	FAQ Bot (GPT-4o)	3,000 queries	\$0.018	\$54	~\$84
GPT-4o Primary TOTAL				\$93 USD	~\$145 AUD
Gemini Hybrid	Contract Checker	600 calls	\$0.04	\$24	~\$37
Gemini Hybrid	Job Checker	600 calls	\$0.008	\$4.80	~\$7.50
Gemini Hybrid	FAQ Bot (Gemini)	3,000 queries	\$0.003	\$9	~\$14
Gemini Hybrid TOTAL				\$37.80 USD	~\$59 AUD
Self-Hosted (All)	All features	4,200 calls	\$0	\$0	~\$5–10 AUD (electricity)

Key insight: Even the all-cloud GPT-4o scenario costs under \$150 AUD/semester. Cost is not the primary differentiator at Internbot's volume. Accuracy and data governance should drive the final decision.

8.2 Performance vs. Cost Trade-Off Matrix

Option	Accuracy	Latency	Cost	Privacy	Ops Burden	Verdict
GPT-5.4 (BIG)	★★★★★	★★★	★★	★★★	★★★★★	Overkill for MVP
GPT-4o (MIDDLE)	★★★★★	★★★★★	★★★	★★★	★★★★★	Recommended — Checkers
Gemini Flash Lite	★★★	★★★★★	★★★★★	★★★★★	★★★★★	Recommended — FAQ
Self-hosted Llama-70B	★★★	★★★	★★★★★	★★★★★	★★	Strong if F1 >= 0.86

9. Implementation Architecture

9.1 Request Flow — Contract & Job Checkers

- Cloud Function receives internship document / job description text from Firestore.
- Prompt template version loaded from Firestore config document (prompt_config/{feature}).
- Request sent to selected model API (RMIT proxy / Gemini / self-hosted vLLM).
- Response parsed: JSON extracted and validated against schema (Zod or similar library).
- If parse fails: retry once with explicit schema reminder appended to user message. If second fail: return safe 503 to the API caller — never surface raw LLM output to the frontend.

- Structured result { decision, confidence, concerns, summary } returned to the calling endpoint. Not persisted (stateless per spec).

9.2 FAQ Bot Request Flow (Firebase-Native)

- Student question received at GET /api/v1/faq?q=...
- Normalised question hash checked against Firestore faq_cache collection (TTL 24h). Cache hit → return immediately.
- Cache miss → question embedded using Gemini text-embedding-004 via Firebase AI Logic SDK.
- Firestore .findNearest() retrieves top-5 chunks from faq_chunks collection.
- Chunks + question injected into FAQ system prompt (Section 4.4). LLM called for generation.
- Response validated: sources array must be present. disclaimer field appended server-side.
- Result cached in faq_cache. Response returned to client.
- Rate limiting: max 10 FAQ queries per student per hour (Firestore rate limit document).

9.3 Model Routing Configuration

Feature	Primary Model	Fallback	Emergency Fallback
Contract Checker	GPT-4o (RMIT proxy)	Self-hosted Llama-70B	Gemini Flash Lite
Job Checker	Gemini Flash Lite	GPT-4o (RMIT proxy)	Self-hosted Llama-70B
FAQ Bot	Gemini Flash Lite	Self-hosted Llama-70B	GPT-4o (RMIT proxy)

Routing is controlled by a Firestore config document (model_routing_config) and can be changed without deployment. Automatic fallback triggers on 5xx or timeout > 30s.

9.4 Self-Hosted vLLM Setup Checklist

Step	Command / Action
Install vLLM	pip install vllm (CUDA 12.4 environment)
Download model	huggingface-cli download meta-llama/Llama-3.1-70B-Instruct
Launch server (4-GPU)	vllm serve meta-llama/Llama-3.1-70B-Instruct --tensor-parallel-size 4 --port 8000
Test endpoint	curl http://localhost:8000/v1/chat/completions -d '{model, messages: [...]}'
Cloud Function config	Set SELF_HOSTED_BASE_URL=http://<workstation-ip>:8000 in Cloud Function env vars
Fallback logic	If self-hosted returns 5xx or no response within 30s → retry on RMIT proxy

10. Testing Plan

Phase 1 — Baseline Benchmark

- Obtain ground-truth dataset: coordinate with Alessio for real contracts, or generate synthetic ones using the test cases in Section 6.2 as a starting point.
- Run all candidate models (GPT-4o, Gemini Flash Lite, Llama-70B self-hosted) against the full Section 6.2 test suite using the baseline prompt (v1.0.0).
- Record all metrics from Section 6.1 into structured JSON benchmark logs.
- Produce the benchmark comparison table and share with supervisor for model selection decision.

Phase 2 — Prompt Iteration

- Analyse failure modes from Phase 1. Identify the top-3 error categories per model (e.g. Rule 1 misclassification, confidence overconfidence, hallucinated concerns).
- Iterate prompt (v1.1.0) to address the identified failures — refine rule descriptions, add edge-case examples, tighten output constraints.
- Re-run benchmark on failing test cases only (targeted regression). Promote best prompt to v2.0.0 as production candidate.
- Document all prompt changes in the version changelog.

Phase 3 — Shadow Testing in Staging

- Deploy production candidate model behind a shadow flag in the staging Cloud Function.
- Real internship submissions (with coordinator permission) are sent to both the shadow model and reviewed by a human in parallel.
- Compare model decision vs. coordinator decision. Log all disagreements for manual review.
- Target: model agrees with coordinator $\geq 88\%$ of the time on real submissions before promotion to production.

Phase 4 — Production Monitoring

- Log every AI call: feature, model, prompt_version, decision, confidence, latency_ms, token counts, parse_success.
- Automated report on a regular cadence: accuracy drift (where GT labels are available), latency trends, cost actuals vs. projections.
- Alert thresholds: P95 latency > 20s or error rate > 5% triggers on-call notification.
- Coordinator samples 20 decisions periodically for quality review. Systematic failures trigger a prompt iteration cycle back to Phase 2.

11. Appendix

11.1 Glossary

Term	Definition
RAG	Retrieval-Augmented Generation — LLM answer grounded in retrieved document chunks.

Term	Definition
F1 Score	Harmonic mean of precision and recall. Single metric balancing both error types.
Precision (Yes)	Of all submissions classified Yes, the proportion that are truly compliant.
Recall (Yes)	Of all truly compliant submissions, the proportion correctly classified Yes.
ECE	Expected Calibration Error — measures how well model confidence matches actual accuracy.
vLLM	Fast LLM inference library with PagedAttention, supporting multi-GPU tensor parallelism.
Tensor Parallelism	Splitting model layers across multiple GPUs to run models larger than single-GPU VRAM.
TTFT	Time To First Token — latency until model starts generating.
PII	Personally Identifiable Information — student names, employer details, salary data.
WIL	Work-Integrated Learning — RMIT's framework for industry placement courses (INTE2710/2711/2708).
Firestore Vector Search	Native approximate nearest-neighbour search in Firestore (GA 2025). Eliminates need for external vector DB.
Firebase AI Logic SDK	Firebase SDK (formerly Vertex AI for Firebase) enabling direct LLM calls from Cloud Functions with no extra auth.

11.2 Benchmark Log Schema

```
{
  "run_id":      "bench-internbot-001",
  "model":      "gpt-4o",
  "model_version": "2024-11-20",
  "prompt_version": "v1.0.0",
  "feature":    "contract_checker",
  "test_id":    "CC-002",
  "ground_truth": "No",
  "model_decision": "No",
  "confidence":  0.99,
  "concerns":   ["Award classification is Health Professionals Award, not IT."],
  "rule_attributed": "Rule 1",
  "latency_ms":  7420,
  "input_tokens": 1842,
  "output_tokens": 156,
  "parse_success": true,
  "timestamp":   "2026-04-15T09:22:11Z"
}
```

11.3 vLLM Hardware Fit Analysis

Model	Params	Approx VRAM (Q4)	Fits in 4× RTX 5090?	Recommended?
Llama-3.1-8B-Instruct	8B	~5 GB	Yes (single GPU)	Baseline only
Llama-3.1-70B-Instruct	70B	~38–40 GB (Q4)	Yes (4-way TP)	Primary candidate
Qwen2.5-72B-Instruct	72B	~41 GB (Q4)	Yes (4-way TP)	Alternate candidate
Llama-3.1-405B-Instruct	405B	~215 GB (Q4)	No — exceeds VRAM	Not feasible